AI Bootcamp

Day 12 Lab: Agentic AI Workflows and Loop Engineering

Bounded Repair Loops, Evidence, Testing, and Safe Application Control

1 Introduction

Day 11 connected an LLM-enabled application to tools. Day 12 asks a harder engineering question: *what controls repeated tool use, and when must the system stop?* In Question 1, you will complete a bounded repair loop for a C++ settlement engine. The Python controller compiles the program, runs tests, observes structured evidence, and chooses to accept, repair, stop, or escalate.

The lab contains:

- **Question 1 (required):** Smart Parking and Bidirectional EV Energy Settlement;
- **Question 2 (optional):** transfer the repair-loop pattern to the Day 8 pathfinding/routing application.

Question 1 must be completed. Question 2 is open-ended and has no Bootcamp submission requirement. Its design may be revised later.

2 Learning Outcomes

After this lab, you should be able to:

1. distinguish a fixed pipeline from a feedback-controlled agent loop;
2. implement, analyse, and justify explicit `ACCEPT`, `REPAIR`, `STOP`, and `ESCALATE` transitions;
3. separate policy, controller, tool, generated candidate, and evidence;
4. design boundary, invalid-input, and business-rule tests;
5. use MCP tools without letting an LLM directly execute arbitrary shell commands;
6. preserve candidates, traces, and test evidence for review;
7. explain why "all visible tests pass" is useful evidence, but not proof of correctness.

3 System Supplied for Question 1

Component	Purpose
C++ workspace	Incomplete C++17 engine; students complete it.
JSON tests	Examples plus the test cases created by students.
Controller	Python controller template; students complete its loop and decisions.
MCP tools	Supplied MCP server and client. Tools compile C++ and run permitted test files.
Revisers	Supplied deterministic reviser and optional Claude Code reviser.
Web demo	Supplied Flask input form and JSON endpoint for simulated carpark readings.
Evidence folders	Audit trace, preserved candidates, and final loop state.

The required path is local and repeatable. It uses prepared repair candidates, not a real LLM. The optional Claude mode uses a paid Claude Code login through the installed `claude` command; no API key is required when that CLI is already authenticated through an eligible account.

3.1 Architecture and trust boundary

```
student/controller
-> MCP client -> supplied MCP server
-> compile_cpp (one permitted source tree)
-> run_tests (one permitted test tree)
<- structured compiler/test evidence
-> decision: ACCEPT | REPAIR | STOP | ESCALATE
-> supplied reviser -> new candidate -> repeat
```

The controller does not give the reviser unrestricted shell access. The MCP server validates paths and exposes two typed capabilities: `compile_cpp(source_path)` compiles one permitted C++ source and returns structured compiler evidence; `run_tests(test_file)` runs one permitted JSON test suite and returns structured results, failures, and a failure signature. The Flask demo calls only the compiled settlement executable. All readings and plates are simulated.

4 Installation and Verification

Use the supplied folder `Q1_Smart_Parking_Agent`. Commands below assume your terminal is inside that folder. Folder names are case-sensitive on some systems. Do not type the prompt symbols shown by your terminal.

4.1 Required software

- Python 3.10 or later;
- a C++17 compiler: Apple Clang on macOS, or MinGW-w64 `g++` on Windows;
- a web browser;
- Internet access only while installing Python packages;
- optional Claude Code CLI for the real-LLM extension.

4.2 macOS: detailed setup

Step M1 — open the bundle folder

In Finder, locate `Q1_Smart_Parking_Agent`. Open Terminal, type `cd` followed by one space, drag the folder into Terminal, and press Return. Then verify the location:

```
pwd
ls
```

You should see `controller`, `mcp_tools`, `workspace`, `tests`, and `requirements.txt`.

Step M2 — verify Python

```
python3 --version
which python3
```

If Python is older than 3.10, install a current Python 3 release before continuing.

Step M3 — verify or install the compiler

```
clang++ --version
```

If the command is missing, install Apple's Command Line Tools:

```
xcode-select --install
```

Complete the installer and open a new Terminal window. Run `clang++ --version` again.

Step M4 — create and activate the virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
python --version
```

The terminal prompt normally begins with `(.venv)` after activation.

Step M5 — install dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Step M6 — verify imports and syntax

```
python -c "import flask, mcp; print(flask.__name__, mcp.__name__)"
python -m compileall controller mcp_tools revisers demo
```

The first command imports both packages and prints their Python module names. Expected output is approximately `flask mcp`. It deliberately avoids nested quotation marks: visually similar "smart

quotes" copied from a PDF, such as U+2019, are not valid Python string delimiters. No output from `compileall` indicates success; normal `Listing` and `Compiling` messages also indicate success.

4.3 Windows PowerShell: detailed setup

This bundle uses MinGW-w64 `g++` on Windows because the supplied compile tool uses GCC/Clang-style arguments. Visual Studio `cl.exe` uses different arguments and is not the default path for this guided lab.

Step W1 — open the bundle folder

In File Explorer, open `Q1_Smart_Parking_Agent`. Click the address bar, copy the full folder path, then in PowerShell run:

```
Set-Location "C:\path\to\Q1_Smart_Parking_Agent"
Get-Location
Get-ChildItem
```

Replace the example path. Keep quotation marks when the path contains spaces.

Step W2 — verify Python

The Windows Python launcher is preferred:

```
py -3 --version
py -3 -c "import sys; print(sys.executable)"
```

If `py` is unavailable, try `python --version`. Install Python 3.10 or later and enable the launcher if neither command works.

Step W3 — verify MinGW-w64 g++

```
g++ --version
Get-Command g++
```

If it is missing, install MSYS2 and its UCRT64 GCC toolchain, then add the toolchain's `bin` directory to the Windows `Path`. Open a new PowerShell window and repeat both checks. Ask the supporting engineer if the classroom machine uses a centrally installed path.

Step W4 — create the virtual environment

```
py -3 -m venv .venv
```

If you used `python` in Step W2, use `python -m venv .venv` instead.

Step W5 — permit local activation if required

First try:

```
.\.venv\Scripts\Activate.ps1
```

If PowerShell reports that script execution is disabled, apply a temporary policy to this PowerShell process only, then activate:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

This does not permanently change the machine policy. Do not use an unrestricted machine-wide policy for this lab.

Step W6 — install and verify dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -c "import flask, mcp; print(flask.__name__, mcp.__name__)"
python -m compileall controller mcp_tools revisers demo
```

4.4 Common setup problems

Symptom	Check
`No module named mcp`	Activate `.venv`; install requirements using that environment's `python`.

`clang++` or `g++` not found	Install the platform compiler; reopen the terminal; verify its path.
PowerShell activation blocked	Use the process-scoped command in Step W5.
Path contains spaces	Put the full path in double quotation marks.
MCP output appears delayed	The server starts as a local subprocess for each tool call; wait several seconds.
Port 5000 already in use	Edit `config/app_config.json`, choose another unoccupied port such as 5050, restart Flask, and open the new address.

5 Question 1 (Required): Smart Parking and EV Settlement

The car park operates a central settlement system for both parking and energy. An EV may import low-priced energy, or export energy to the station when the vehicle, station, and owner all permit vehicle-to-grid (V2G) operation. Therefore the final amount may be positive, zero, or negative. A negative amount is a credit owed to the driver; it is not automatically an error.

5.1 Business rules

Read `specs/settlement_rules.txt`; it is the authoritative lab policy. The main rules are:

1. accepted vehicle types are Internal Combustion Engine car (`ICE_CAR`), `EV_CAR`, and `MOTORCYCLE`;
2. for this simplified exercise, one settlement record covers at most one 24-hour period, so parking time is 0–1440 minutes inclusive; a production system would split or aggregate multi-day stays according to its policy;
3. up to 15 minutes is free; each later fee uses started hours and a daily cap;
4. energy values and tariffs cannot be negative;
5. energy cents use integer half-up rounding;
6. export requires an EV, vehicle and station capability, owner opt-in, and remaining SOC at least equal to minimum departure SOC;
7. $net = parking + import - export$;
8. net > 0 is `PAY`, net $= 0$ is `ZERO`, and net < 0 is `CREDIT`.

5.2 Your required work

You must complete all three student-owned items:

1. implement the missing C++ validation and calculation functions in the supplied workspace source;
2. add meaningful cases to the student JSON test file;
3. complete `choose_next_action` and `run_loop` in `controller/agent_controller_template.py`.

Do not modify the supplied MCP server, test runner, deterministic candidates, or web interface during the required guided exercise.

5.3 Part A — understand input and output

The C++ executable receives eleven command-line values in this order:

```
parking_minutes vehicle_type import_wh import_rate_cents_per_kwh
export_wh export_rate_cents_per_kwh vehicle_v2g station_v2g
owner_opt_in soc_after_export minimum_departure_soc
```

It writes exactly one JSON object. A successful credit may look like:

```
{"status":"ok","parking_fee_cents":350,"import_cost_cents":50,
  "export_credit_cents":600,"net_amount_cents":-200,
  "settlement":"CREDIT"}
```

An invalid export writes `status:error` and a useful error string. The test runner compares structured fields, not prose generated by an LLM.

5.4 Part B — complete the C++ engine

Open `workspace/settlement_engine.cpp`. Find every `TODO`. Work from the specification, not only from the example outputs.

Implementation reminders:

- for a started-hour calculation, integer arithmetic can compute a ceiling;
- use a wide integer type before multiplying Wh by cents/kWh;
- half-up rounding for nonnegative integers can be implemented before division;
- return validation failures before calculating a settlement;
- do not use floating-point comparisons for monetary cents.

5.5 Part C — design tests before trusting the loop

Inspect `tests/example_tests.json`; then add cases to `tests/student_tests.json` using the same JSON format. Include at least:

1. one parking boundary (for example, 15 versus 16 minutes);
2. one vehicle-type or daily-cap case;
3. one invalid V2G permission combination;
4. one departure-SOC violation;
5. one exact-zero or negative settlement;
6. one energy rounding boundary.

Each test must have a descriptive `name`, complete `input`, and an `expected` object. For an error case, use `error_contains` to match the important part of the diagnostic.

Example skeleton:

```
[
{
  "name": "describe the rule or boundary",
  "input": {
    "parking_minutes": 15,
    "vehicle_type": "EV_CAR",
    "import_wh": 0,
    "import_rate_cents_per_kwh": 0,
    "export_wh": 0,
    "export_rate_cents_per_kwh": 0,
    "vehicle_v2g": false,
    "station_v2g": false,
    "owner_opt_in": false,
    "soc_after_export": 70,
    "minimum_departure_soc": 40
  },
  "expected": {"status":"ok", "net_amount_cents":0,
    "settlement":"ZERO"}
}
]
```

Validate the JSON after editing:

macOS

```
python -m json.tool tests/student_tests.json
```

Windows PowerShell

```
python -m json.tool .\tests\student_tests.json
```

If the JSON is valid, this command prints the same data in a consistently indented format. It does not execute the settlement tests and does not prove that the expected values are correct. Invalid JSON instead produces a parsing error with an approximate line and column.

5.6 Part D — complete the controller

Open `controller/agent_controller_template.py`. Implement a bounded observe–decide–act loop. Do not replace it with "repeat until tests pass."

Your decision function must implement this policy:

Observation	Action	Reason
Compilation failed and budget remains	`REPAIR`	candidate cannot be tested
All selected tests passed	`ACCEPT`	stated acceptance condition met
Iteration budget exhausted	`STOP`	bounded resource use
Same failure signature seen again	`ESCALATE`	loop is not making useful progress
Tests failed, budget remains	`REPAIR`	evidence requests another revision

The loop must also:

- preserve every candidate and its hash;
- compile before running tests;
- never test a failed compilation;
- record structured trace evidence;
- stop on a repeated candidate hash;
- write `state/final_state.json` for every normal exit.

Suggested pseudocode:

```
initialise bounded state
save candidate 0
while true:
read and hash candidate
if hash repeated: escalate and break
compile through MCP
if compile passed: test through MCP
action = choose_next_action(state, compile_result, test_result)
write trace
if ACCEPT, STOP, or ESCALATE: update final state; break
ask selected reviser for a complete revised source
increment iteration and preserve the candidate
write final state
```

5.7 Part E — run the deterministic loop

The deterministic reviser makes the classroom result repeatable. It lets you debug the controller without waiting for a real LLM or consuming tokens.

Running the unfinished student template produces `NotImplementedError: Complete run_loop`; this is expected until you replace its TODO placeholders. A complete released reference controller is available at `instructor/agent_controller_instructor.py`. Use it when directed, or to verify the MCP tools, reviser, C++ candidate, and tests independently of your unfinished controller. You must still complete the student template and be able to justify its decisions and stop conditions.

macOS

```
source .venv/bin/activate
python controller/agent_controller_template.py \
--reviser simulated --tests tests/example_tests.json --max-iterations 4
```

Windows PowerShell

```
.\.venv\Scripts\Activate.ps1
python .\controller\agent_controller_template.py `
--reviser simulated --tests tests/example_tests.json --max-iterations 4
```

The PowerShell continuation character is a backtick at the end of the line. It must be the final character; trailing spaces break continuation. Alternatively, put the entire command on one line.

With a correct controller and the untouched deterministic candidates, the guided trace should progress approximately as follows:

```
Iteration 0: compile=PASS, tests=0/5, decision=REPAIR
Iteration 1: compile=PASS, tests=4/5, decision=REPAIR
Iteration 2: compile=PASS, tests=5/5, decision=ACCEPT
Final status: PASSED -- all_tests_passed
```

Exact supporting messages may vary with the MCP package and operating system.

To run the released reference controller instead, substitute this script path:

```
instructor/agent_controller_instructor.py
```

5.8 Part F — inspect evidence

Do not stop at the green final line. Inspect what the agent actually did.

macOS

```
ls candidates
cat state/final_state.json
tail -n 3 logs/trace.jsonl
```

Windows PowerShell

```
Get-ChildItem .\candidates
Get-Content .\state\final_state.json
Get-Content .\logs\trace.jsonl -Tail 3
```

Answer these questions in your submission notes:

1. Which observation caused each repair?
2. What defect remained in candidate 1?
3. Which two independent conditions prevent an endless loop?
4. Why is a candidate hash different from a failure signature?
5. What could still be wrong after the five visible tests pass?

5.9 Part G — run your student tests

Run the controller or supplied solution path against your own file. During development, the instructor may provide a known-good controller separately if you need to isolate C++ and test-design problems from controller problems.

macOS

```
python controller/agent_controller_template.py \
--reviser simulated --tests tests/student_tests.json --max-iterations 4
```

Windows PowerShell

```
python .\controller\agent_controller_template.py ‘
--reviser simulated --tests tests/student_tests.json --max-iterations 4
```

The instructor or supporting engineer may then apply additional acceptance cases not visible while you develop. A failure is evidence to diagnose, not a reason to keep increasing the loop limit.

5.10 Part H — run the supplied web simulator

First ensure the current C++ candidate compiles and passes. Start Flask in a separate terminal with the same virtual environment.

macOS

```
source .venv/bin/activate
python demo/app.py
```

Windows PowerShell

```
.\.venv\Scripts\Activate.ps1
python .\demo\app.py
```

At startup, Flask reads `config/app_config.json` and prints the active address. The default is http://127.0.0.1:5000. If that port is occupied, stop Flask, change only the `port` value in the configuration file (for example, to 5050), restart it, and open the printed address. Enter simulated parking, charging, tariff, permission, and SOC readings. Verify at least one `PAY`, one `CREDIT`, and one rejected V2G scenario. Stop the server with Ctrl+C.

The disabled example plate is for interface context only and is not passed to the settlement engine. A production design would require identity, consent, meter integrity, tariff versioning, and transaction approval controls beyond this coding exercise.

5.11 Part I (Optional) — real Claude Code revision

This extension is not required. Use it only if the `claude` CLI is installed and authenticated. It may consume the usage allowance of the signed-in account. Unlike the simulated reviser, Claude reads the current C++ source, written rules, and compiler/test evidence and returns genuinely generated C++.

5.11.1 Verify installation, login, and non-interactive access

First verify the installed command:

```
claude --version
```

The version command proves installation but does not prove that the stored OAuth token is current. From the project directory, start Claude Code:

```
claude
```

If required, enter `/login`, complete browser authentication, confirm `Login successful`, and exit with `/exit`. Then verify a real non-interactive model request:

```
claude -p "Reply only with OK" \
--max-turns 1 --output-format text
```

Expected output is `OK`. If the command reports `401 OAuth access token has expired`, repeat `/login`. Do not begin the repair experiment until this preflight request succeeds.

5.11.2 Controlled real-LLM repair experiment

Use a known-good instructor or completed C++ engine and the fuller working test file. Preserve the correct source first:

```
cp workspace/settlement_engine.cpp \
workspace/settlement_engine_correct.cpp
```

Run the correct version once:

```
python instructor/agent_controller_instructor.py \
--reviser claude --tests tests/student_tests.json \
--max-iterations 4
```

If all 12 tests pass, the controller chooses `ACCEPT` immediately and does not call Claude. Next, intentionally change the grace-period condition in `workspace/settlement_engine.cpp` from:

```
if (input.parking_minutes <= 15) return 0;
```

to:

```
if (input.parking_minutes <= 10) return 0;
```

Run the same command again. A successful demonstration should be similar to:

```
Iteration 0: compile=PASS, tests=11/12, decision=REPAIR
Iteration 1: compile=PASS, tests=12/12, decision=ACCEPT
Final status: PASSED -- all_tests_passed
```

The first test run exposes the 15-minute boundary defect. The controller sends the current source, specification, and structured failure evidence to Claude. Claude proposes a complete revised C++ file; the controller preserves it, compiles it, runs the tests again, and accepts it only when the evidence meets the stated condition.

Inspect the actual change:

```
diff -u workspace/settlement_engine_correct.cpp \
workspace/settlement_engine.cpp
diff -u candidates/candidate_0.cpp candidates/candidate_1.cpp
tail -n 3 logs/trace.jsonl
```

5.11.3 Two different loop limits

The corrected Claude adapter allows up to three internal Claude turns for one repair request. The command-line option `--max-iterations 4` limits the outer Python repair loop. These are different budgets:

- **Claude `--max-turns 3`:** model turns inside one revision request;

• **controller `--max-iterations 4`:** compile–test–repair cycles.

The adapter reports Claude's exit code, stdout, and stderr when a request fails. Claude does not receive permission through this adapter to redesign the MCP server, modify tests, add network access, or run arbitrary shell commands.

Before running real-LLM mode, copy your work. LLM output is nondeterministic and may return invalid or unnecessary changes. Never remove the maximum-iteration, repeat-detection, path, or evidence controls merely to make the demonstration continue.

5.12 Submission checklist for Question 1

Submit the files specified by the instructor. At minimum, be ready to show:

- completed `workspace/settlement_engine.cpp`;
- completed `controller/agent_controller_template.py`;
- expanded `tests/student_tests.json`;
- `state/final_state.json` and the relevant trace;
- successful web demonstrations of pay, credit, and safe rejection;
- short answers to the five evidence questions in Part F.

6 Question 2 (Optional): Pathfinding Repair Loop with a Real LLM

This extension reuses the algorithmic core of the Day 8 AI Review pathfinding exercise. It has no required Bootcamp submission. Do not rebuild the complete web visualisation, authentication, Docker, or Jenkins environment. Concentrate on transferring the bounded repair-loop pattern to a second application.

6.1 Objective

Start from a working pathfinding implementation that reads `Node_Info.txt` and `Graph_Path.txt`, computes a shortest route, and has repeatable automated tests. Introduce one controlled defect, ensure a test exposes it, and use a real LLM revision inside a bounded loop to propose and verify a repair.

```
preserve baseline -> introduce defect -> run syntax check and tests
-> collect failure evidence -> ask LLM for revised pathfinder
-> preserve candidate -> test again -> accept, stop, or escalate
```

6.2 Reduced project scope

Reuse only:

- the node and graph-path data files;
- the map loader and shortest-path implementation;
- the pathfinding verification tests.

The Day 8 web interface, drawing, map editing, authentication, Docker, and CI requirements are outside this optional exercise.

6.3 Step 1 — establish a baseline

Run the original pathfinding tests and preserve the known-good source. Record the command, pass count, and source hash. The baseline must pass before you introduce a defect; otherwise later evidence cannot be attributed confidently.

6.4 Step 2 — introduce one controlled defect

Choose one defect that has a clear expected behaviour:

1. treat an undirected edge as one-way;
2. use coordinate distance instead of the `Distance` field;
3. mishandle `start == end`;
4. fail to report an invalid start or end node;

5. return an empty path without a clear unreachable result;
6. stop Dijkstra's algorithm when a node is first discovered instead of when its shortest distance is finalised.

Preserve the defective source as candidate 0. Do not introduce several unrelated defects in the first experiment.

6.5 Step 3 — create evidence that exposes the defect

Identify an existing failing test or add one regression test with explicit input and expected output. For example, a one-way-edge defect requires a reverse-route request. If every test passes, the controller has no evidence that the source is wrong and should accept it unchanged.

Useful cases from Day 8 include:

- start equals end;
- invalid start or end node;
- disconnected graph or isolated node;
- cycles and multiple possible routes;
- two equal-distance shortest routes;
- edge cost different from straight-line coordinate distance.

6.6 Step 4 — adapt the loop and tools

The Python pathfinding target does not need C++ compilation. Replace that observation with a Python syntax check and test execution. Suitable narrow typed capabilities are:

```
check_python(source_path)
run_pathfinding_tests(test_path)
```

The first tool may use `python -m py_compile`; the second may invoke `pytest` or the supplied verification script. Both should return structured success, stdout, stderr, and failure information. Restrict permitted source and test paths rather than exposing a general shell.

The controller policy remains the same:

- syntax failure or failed tests with budget remaining: `REPAIR`;
- all selected tests pass: `ACCEPT`;
- repeated candidate or repeated failure: `ESCALATE`;
- iteration budget exhausted: `STOP`.

6.7 Step 5 — run and inspect a real-LLM repair

Complete the Claude login and non-interactive preflight from Question 1. Run the pathfinding controller, preserve every candidate, and inspect the source diff and trace. A possible result is:

```
Iteration 0: syntax=PASS, tests=7/8, decision=REPAIR
Iteration 1: syntax=PASS, tests=8/8, decision=ACCEPT
Final status: PASSED -- all_tests_passed
```

Exact output is nondeterministic. Claude proposes the repair; tests and the controller decide whether the proposal satisfies the stated acceptance condition.

6.8 Step 6 — independent verification

Run at least one holdout map or request that was not included in the evidence sent to Claude. This checks whether the repair generalises beyond the single failing example. Passing the regression test is necessary evidence, not proof of general correctness.

6.9 Optional evidence to retain

If you continue this exercise after class, retain:

- known-good, defective, and LLM-repaired source files;
- the regression test and one holdout test;
- controller trace and final state;
- a concise diff and explanation of the acceptance decision.

7 Safety and Engineering Lessons

This lab intentionally separates permissions and responsibilities:

- policy text defines what "correct" means;
- the C++ engine performs deterministic settlement arithmetic;
- test files encode selected claims about policy;
- MCP exposes narrow compile/test capabilities;
- the reviser proposes code but does not decide when it is trustworthy;
- the controller owns state transitions and budgets;
- traces support later review;
- a human owns deployment and consequential approval.

An agentic loop is not made safe merely by adding retries. Its engineering value comes from explicit state, bounded action, typed tools, evidence, preservation, and well-defined stop or escalation conditions.

8 End-of-Lab Reflection

Be prepared to discuss:

1. Is the deterministic classroom loop genuinely agentic under a strict engineering definition? Explain what is dynamic and what is fixed.
2. Would replacing MCP with direct process calls change the loop logic? What interoperability, cost, and security trade-offs would change?
3. Which decision must remain outside the LLM?
4. How could test generation and implementation collude to pass while both misunderstand the specification?
5. What additional controls would be required before handling real vehicles, meters, payments, or automatically publishing repaired application code?